STAT 547: Homework

Jenna Melanson

2026-01-30

Exercise 1

a) A Stan program for the simple pharmacokinetic model

```
// ---- simple_pharmacokinetic ----

functions {
  vector simplepharm(real t, vector u, real ka, real CL, real Vcent) {
    real ugut = u[1];
    real ucent = u[2];

    vector[2] du_dt;
    du_dt[1] = - ka * ugut;
    du_dt[2] = ka * ugut - CL / Vcent * ucent;

    return du_dt;
  }
}

data {
  int<lower=1> N; // number of observations
  vector[N] y; // observed concentrations (e.g., c(t))
  array[N] real timepoints; //observation timepoints
  vector[2] u0; // starting values for ugut, ucent
  real t0;
}


parameters {
  // I assume here that absorption must be positive or 0 (e.g., we do not account for
  // reverse absorption in the simple model), and that similarly clearance must be
  // positive or 0 (no reabsorption from the periphery)
  real<lower=0> ka;
  real<lower=0> CL;
  real<lower=0> Vcent;
  real<lower=0> sigma;
}

transformed parameters {
  array[N] vector[2] u = ode_rk45(simplepharm, u0, t0, timepoints, ka, CL, Vcent);
}
```

```
model {
CL ~ lognormal(log(10), 0.25);
Vcent ~ lognormal(log(35), 0.25);
ka ~ lognormal(log(2.5), 1);
sigma ~ normal(0, 1);

for (t in 1:N) {
    y[t] ~ lognormal(log(u[t][2] / Vcent), sigma);
  }
}


generated quantities {
  array[N] real ucent_pred;
  vector[N] log_lik;
  for (t in 1:N){
    ucent_pred[t]= lognormal_rng(log(u[t][2] / Vcent), sigma);
    log_lik[t] = lognormal_lpdf(y[t] | log(u[t][2] / Vcent), sigma);
  }
}
```

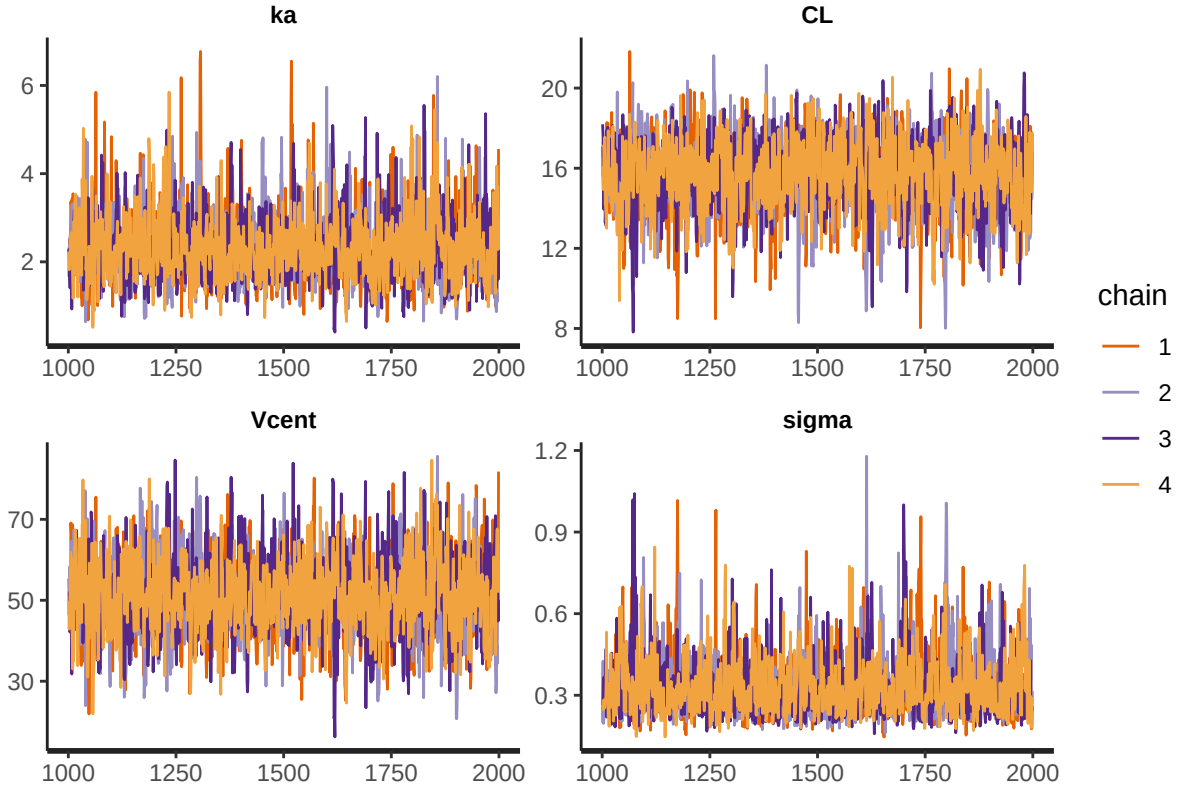
b) Fitting the model

I fit the model with 4 chains of 2000 iterations each (1000 warm up, 1000 sampling). There were no divergent transitions, and the model fit produces the following summary statistics and diagnostics for our parameters of interest:

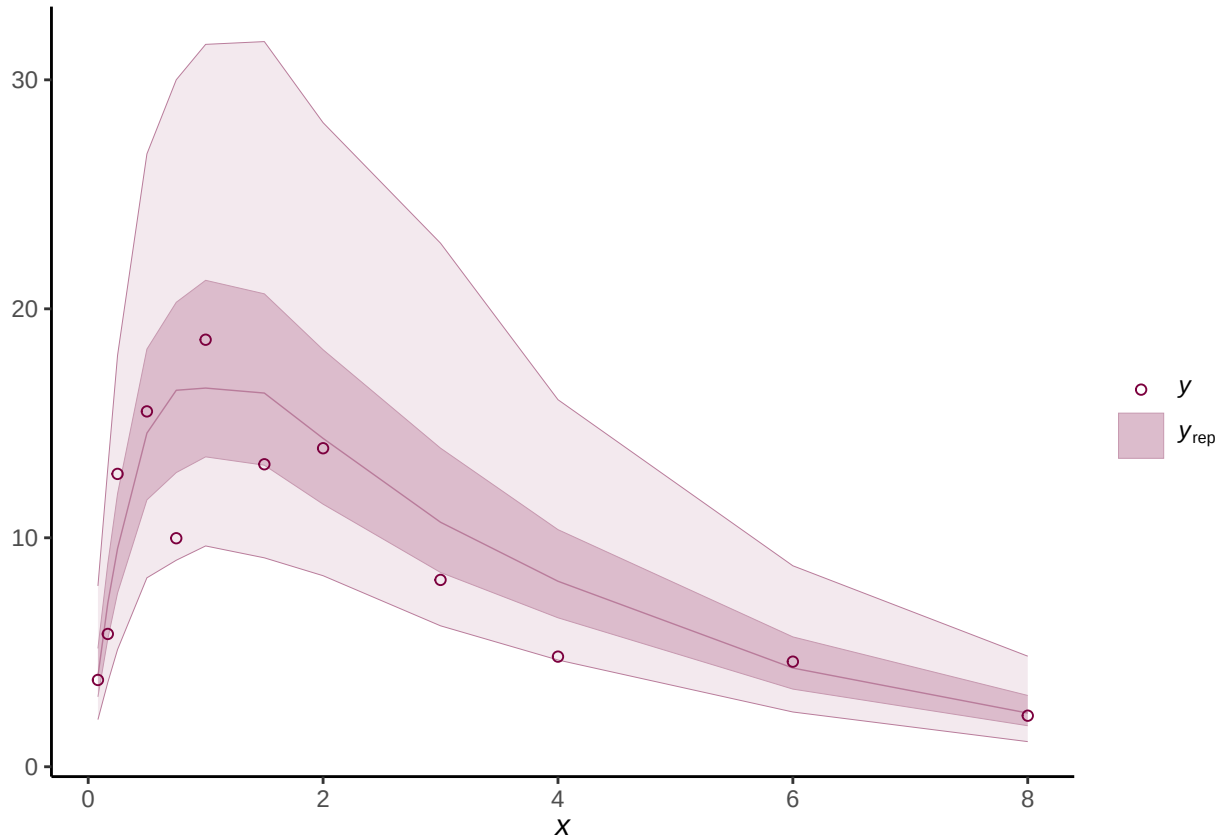
Table 1:

	mean	se_mean	sd	2.5%	25%	50%	75%	97.5%	n_eff	Rhat
ka	2.308	0.018	0.752	1.090	1.810	2.213	2.709	4.103	1746.892	1.001
CL	15.596	0.050	1.818	11.780	14.472	15.697	16.827	18.838	1334.353	1.002
Vcent	50.618	0.252	9.440	32.318	44.439	50.458	56.666	69.620	1401.407	1.003
sigma	0.333	0.003	0.110	0.193	0.256	0.312	0.383	0.623	1003.312	1.002

The effective sample size (n_eff) is greater than 1000 for all four parameters. While it is difficult to say what precise ESS is "necessary," from an inferential standpoint, the Monte Carlo Standard Errors (se_mean) are quite small in comparison to the posterior standard deviations (sd), suggesting that we have drawn enough samples to reduce the persistent variance of our chain. Values for Rhat are all very close to 1 (<1.01), suggesting that the chains have converged (i.e., the chains have come reasonably close to the stationary distribution and are sampling from the same region of parameter space; the non-stationary variance has approached zero). We can additionally look at some traceplots to verify that the chains are mixing well visually, and to check for any signs of periodicity:



Finally, lets produce some posterior predictive (retrodictive) checks to assess whether our model can make accurate "predictions" and quantify uncertainty around those predictions. Below I show the original observed values of y ($C(t)$), overlaid by estimates of y_rep from the `generated quantities` block.



There appear to be some oscillations (maybe noise/measurement error…) that are not accounted for by the model's central tendency. In addition the model does not capture the tail behaviour very well—the observed concentrations drop off more rapidly around t = 3-4 than anticipated by the model. Fortunately, the model does a good job of quantifying its uncertainty to accomodate these deviations, with a large posterior predictive confidence interval encompassing > 90% of the data. But it would seem that there may be some biological processes unaccounted for by the two-compartment model.

c) Incorporating the peripheral compartment

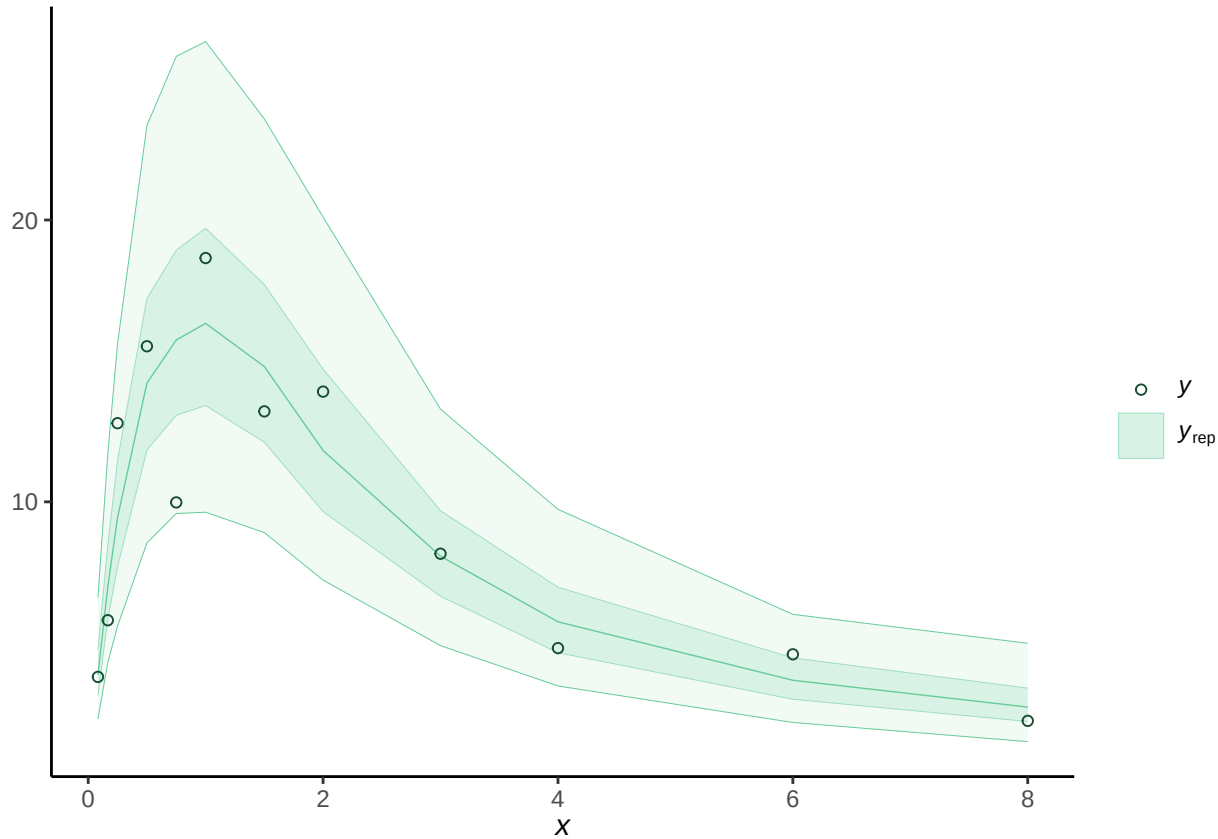
Ahh, a more sophisticated model! How convenient!

We can fit this model using the following Stan program:

```
// ---- peripheral_pharmacokinetic ----

functions {
  vector peripharm(real t, vector u, real ka, real CL, real Vcent, real Q, real Vperi) {
    real ugut = u[1];
    real ucent = u[2];
    real uperi = u[3];

    vector[3] du_dt;
    du_dt[1] = - ka * ugut;
    du_dt[2] = ka * ugut - ((CL+Q)/Vcent) * ucent + (Q / Vperi)*uperi;
    du_dt[3] = (Q/Vcent) * ucent - (Q/Vperi) * uperi;

    return du_dt;
```

```
  }
}

data {
  int<lower=1> N; // number of observations
  vector[N] y; // observed concentrations (e.g., c(t))
  array[N] real timepoints; //observation timepoints
  vector[3] u0; // starting values for ugut, ucent
  real t0;
}


parameters {
  // I assume here that absorption must be positive or 0 (e.g., no reverse
  // absorption to gut), and that both peripheral and elimination clearance
  // must be positive or 0
  real<lower=0> ka;
  real<lower=0> CL;
  real<lower=0> Vcent;
  real<lower=0> Q;
  real<lower=0> Vperi;
  real<lower=0> sigma;
}

transformed parameters {
  array[N] vector[3] u = ode_rk45(peripharm, u0, t0, timepoints, ka, CL, Vcent, Q, Vperi);
}


model {
CL ~ lognormal(log(10), 0.25);
Vcent ~ lognormal(log(35), 0.25);
ka ~ lognormal(log(2.5), 1);
Q ~ lognormal(log(15), 0.5);
Vperi ~ lognormal(log(105), 0.5);
sigma ~ normal(0, 1);

for (t in 1:N) {
    y[t] ~ lognormal(log(u[t][2] / Vcent), sigma);
  }
}


generated quantities {
  array[N] real ucent_pred;
  vector[N] log_lik;
  for (t in 1:N){
    ucent_pred[t]= lognormal_rng(log(u[t][2] / Vcent), sigma);
    log_lik[t] = lognormal_lpdf(y[t] | log(u[t][2] / Vcent), sigma);
  }
}
```

Assuming $u_{peri}(0) = 0$, our model produces the following posterior predictions:



The central tendency of the new model appears to more accurately match the observed data, particulary around t = 3-4. The predictive confidence intervals are also narrower, but still encompass all of the datapoints.

But which model is really better? We can perform leave-one-out cross validation to check.

```
## Warning: Some Pareto k diagnostic values are too high. See help('pareto-k-diagnostic') for details.
```

```
##
## Computed from 4000 by 12 log-likelihood matrix.
##
##          Estimate  SE
## elpd_loo    -29.1 2.7
## p_loo         2.5 0.4
## looic        58.2 5.4
## ------
## MCSE of elpd_loo is NA.
## MCSE and ESS estimates assume MCMC draws (r_eff in [0.3, 1.0]).
##
## Pareto k diagnostic values:
##                           Count Pct.    Min. ESS
## (-Inf, 0.7]   (good)     11    91.7%   595
##    (0.7, 1]   (bad)       1     8.3%   <NA>
##    (1, Inf)   (very bad)  0     0.0%   <NA>
## See help('pareto-k-diagnostic') for details.
```

```
##
## Computed from 4000 by 12 log-likelihood matrix.
```

```
##
##          Estimate  SE
## elpd_loo    -27.4 2.9
## p_loo         2.9 0.7
## looic        54.8 5.7
## ------
## MCSE of elpd_loo is 0.1.
## MCSE and ESS estimates assume MCMC draws (r_eff in [0.5, 1.2]).
##
## All Pareto k estimates are good (k < 0.7).
## See help('pareto-k-diagnostic') for details.
```

Ahh, but of course you would not make this *too* easy. One of the datapoints has a high Pareto k value (k = 0.855) when we perform leave-one-out cross validation with the simpler model, meaning that we cannot trust our overall estimate of `elpd_loo` for that model. (I might argue that this alone suggests that the model including peripheral absorption is a better fit to the data, but presumably the *most correct* approach would be to compute `elpd_loo` manually for this model.)

To do this I will fit the model to the data *without* the problematic datapoint (y[5]) and compute the log-likelihood of the held out value based on parameter posteriors from the holdout model:

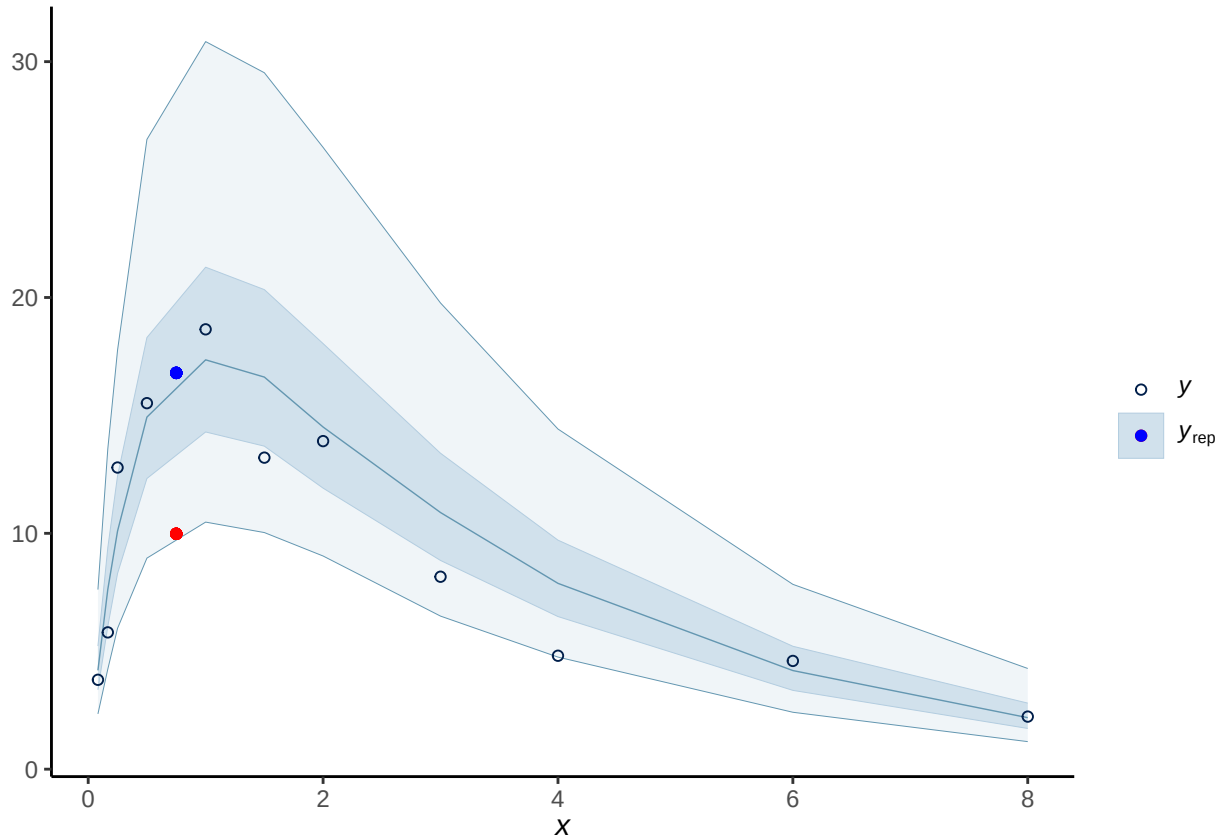
$$\widehat{\text{elpd}}_5 = \log\left(\frac{1}{S}\sum_{s=1}^{S} p\left(y_5 \mid \theta_{-5}^{(s)}\right)\right)$$

where S is the number of posterior samples. We can compute this as:

$$\widehat{\text{elpd}}_5 = \text{log_sum_exp}\left(\log p\left(y_5 \mid \theta_{-5}^{(s)}\right)\right) - \log S$$

using log likelihood values from the `generated quantities` block.

First, I'll refit the model and perform another posterior predictive check, just to ensure that I've coded it out properly.



The red point is the one which we have held out (unsurprising that this has a high Pareto k value…it's a bit of an outlier!). The filled blue point is the posterior median of the predictions for the held-out timepoint (t = 0.75). The prediction doesn't fall perfectly on the "smoothed" curve, but that's not surprising given that the smooth curve is based on only 11 timepoints. If I created a fine-scale grid of timepoints at which to compute the predicted values, the curve would look less choppy and I suspect that my predicted holdout value would be pretty spot-on. Sanity check complete!

Next we can compute that $\widehat{\text{elpd}}_5$ = -3.711, using log-likelihood values from the `generated quantities` block and the equation above.

This is great but it's only one value! Next we must add this value to the other PSIS estimates for the model, giving us a total `elpd_loo` value for the simple model of -29.106.

Based on a bit of reading, it seems that we should also perform this "replacement" with the exact value for the second model, if we want the comparison to be valid.

If we repeat the same steps for the model including the peripheral compartment, we arrive at `elpd_loo` = -27.43. (I note that these are identical to the PSIS estimated values, so it would seem that the high Pareto K value did not impact the `elpd` estimate for y[5] too substantially.)

Finally we can compute that `elpd_diff` = 1.676 with standard error 1.734.

So that was a whole lot of work to show that the models have basically similar predictive power; the more complex model has a slightly better `elpd_loo`, but because this difference is smaller than the standard error on the estimates, we could either report both models and their respective weights, or just the simpler (more parsimonious) model—at least this is what I tend to see in the biological literature. However, I'm curious to know what a proper statistician's perspective would be…?

Exercise 2

As a disclaimer, the sampler diagnostic checks for parts (b-d) look quite bad. Despite many attempts at debugging, I don't see any errors in my sampler code (maybe I've missed some). I suspect this is an issues with the values for `eps`, `L`, the small number of iterations, and the way they interact with the target distribution. Increasing any of these parameters singly or in combination rescues the behaviour, as seen in parts (e-f). Similarly, if I replace your covariance matrix with a 5x5 identity matrix, the diagnostics are considerably improved (although not perfect). Maybe this was the intention of the question (to show that some `L` produce pathogenic chains, even for a seemingly simple target), but you seemed to suggest in office hours that it should work.

Anyways, I was unhappy with the diagnostics and wanted to verify that the issue was not elsewhere in my code (e.g., incorrectly computing the summary statistics), so I present all of the diagnostics for a sampler using `L` = 80 as well. These can be found at the end of the question.

a) Write a function to run "static" HMC

Because we are working here with Gaussian distributions, it's possible to derive an analytical solution for `grad_log_p`. Starting from the log probability density for a multivariate Gaussian:

$log(\pi) = -\frac{k}{2}log(2\pi) - \frac{1}{2}log(|\Sigma|)) - \frac{1}{2}(x-\mu)^T\Sigma^{-1}(x-\mu)$

$\frac{d}{dx}log(\pi) = -\Sigma^{-1}(x-\mu)$

The log density functions I will supply to the sampler are effectively just wrapper functions on `dnorm` (base R) and `dmvnorm` from the package `mvtnorm`:

```
standard_gradient = function(x0){
  grad = -x0
  return(grad)

}

standard_lpdf = function(x0){
  return(dnorm(x0, 0, 1, log = TRUE))
}

multivariate_gradient = function(x0, # length k vector -- where are we?
                             mu = rep(0,5),
                             sigma = matrix(c(2.82, 2.71, 0.18, 2.42, -1.87,
                                              2.71, 7.35,-4.02,-1.17,-1.05,
                                              0.18,-4.02,8.85,1.64,3.81,
                                              2.42,-1.17,1.64,5.94,-3.95,
                                              -1.87,-1.05,3.81,-3.95,6.5), nrow = 5, ncol = 5)
                             ){
  grad = as.numeric(-solve(sigma) %*% (x0-mu))
  return(grad)

}

multivariate_lpdf = function(x0,
                         mu = rep(0,5),
                             sigma = matrix(c(2.82, 2.71, 0.18, 2.42, -1.87,
                                              2.71, 7.35,-4.02,-1.17,-1.05,
                                              0.18,-4.02,8.85,1.64,3.81,
                                              2.42,-1.17,1.64,5.94,-3.95,
                                              -1.87,-1.05,3.81,-3.95,6.5), nrow = 5, ncol = 5)
```

```
){
  return(dmvnorm(x0, mu, sigma, log = TRUE))
}
```

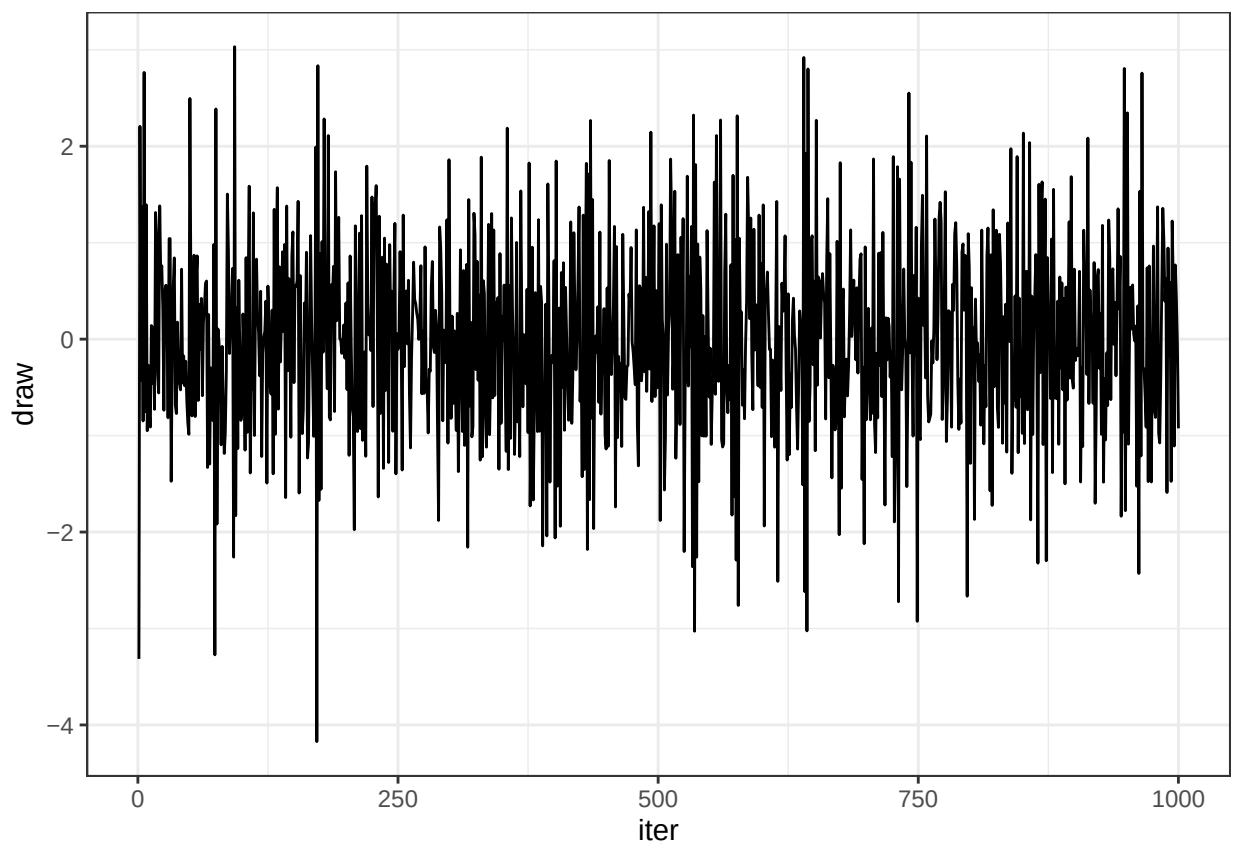
Then we can commence with writing the HMC sampler itself! I am looking here to the code from Neal (2011); frankly this feels like cheating because the code is basically already written??

```
HMC = function(log_p,
               grad_log_p,
               theta0,
               n_iter,
               eps,
               L){
  q = theta0
  draws = matrix(NA, nrow = n_iter, ncol = length(theta0))
  numgrad = numeric(n_iter)

  for (n in 1:n_iter){
  numgrad[n] = 0
  start_q = q
  p = rnorm(length(q),0,1)
  start_p = p

  ################# Leapfrog ##############################
  # half step for momentum at the beginning
  p = p + eps*grad_log_p(q) / 2
  numgrad[n] = numgrad[n] + 1

  for (i in 1:L){
  # full step for position
    q = q + eps * p
  # full step for momentum, except at end of trajectory
    if (i!=L) {
      p = p + eps*grad_log_p(q)
      numgrad[n] = numgrad[n] + 1
      }
  }

  # half step for momentum at the end.
  p = p + eps*grad_log_p(q) / 2
  numgrad[n] = numgrad[n] + 1
  p = -p # for detail balance, not strictly necessary
  ##########################################################


  ################## Metropolis Acceptance ################
  # evaluate potential and kinetic energies
  start_U = -log_p(start_q)
  start_K = sum(start_p^2) / 2
  proposed_U = -log_p(q)
  proposed_K = sum(p^2) / 2

  # accept or reject the state at end of trajectory
  if (runif(1) < exp(start_U-proposed_U+start_K-proposed_K)){
```

```
    draws[n,] = q
    } else{
      q = start_q
      draws[n,] = q
    }
  ######################################################
  }
  return(list(draws, numgrad))
}
```

I will call this function on a standard normal distribution to test its performance. I'll use `n_iter = 1000`, `eps = 0.1`, `L = 20`, and `theta0 = 10`.

Discarding the first 500 iterations as warm-up, the Monte Carlo mean estimate is -0.002 and the variance is 1.11. Let's check out the traceplot.



b) Sampling from a multivariate normal

In my `log_p` and `grad_log_p` functions, I have assumed that the mean is located at {0, 0, 0, 0, 0}. For the stated parameter values (`eps` = 0.05, `L` = 10, 1000 iterations), I run the sampler and compute summary statistics using the package `posterior`:

variable	mean	median	sd	q5	q95	mcse_mean
X1	0.760	0.629	1.389	-1.290	2.933	0.527
X2	-1.189	-1.012	2.541	-5.372	2.680	0.975
X3	1.502	1.378	1.605	-0.835	4.361	1.137

variable	mean	median	sd	q5	q95	mcse_mean
X4	2.484	2.474	1.500	0.208	5.125	0.401
X5	-1.255	-1.319	1.772	-4.125	1.941	1.126

The mean estimates don't look fantastic, but the confidence bounds at least overlap with 0 for each variable. The MCSE means are fairly substantial in comparison to the posterior standard deviations, which might suggest that we need more iterations to reduce the variance of the estimator.

c) Convergence diagnostics

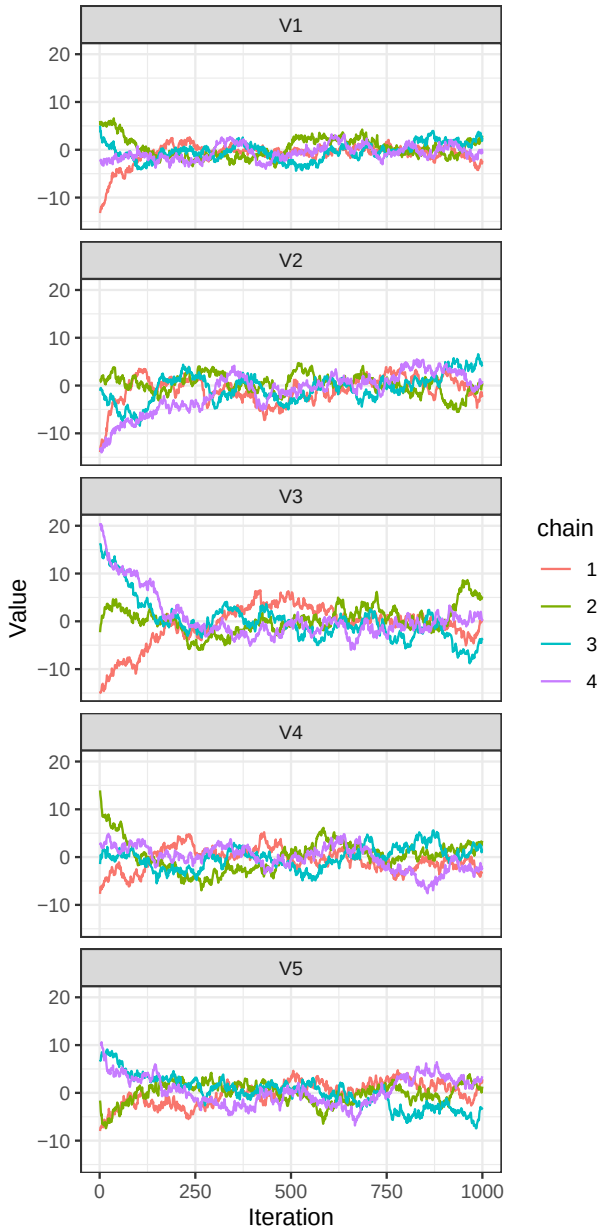
Dimension	Rhat	ESS
V1	1.058	45.215
V2	1.306	11.378
V3	1.167	22.741
V4	1.535	7.579
V5	1.386	8.837

When we compute $\hat{R}$ and bulk effective sample size (ESS) using multiple chains, we can see that $\hat{R}$ is consistently larger than 1.01, indicating that the chains have not mixed well. The effective sample size is quite low in comparison to our number of iterations (2000 sampling iterations total if we combine the four chains). To me this suggests that we are not getting a lot of information per sample, possibly because the chains are highly autocorrelated. This would cause low ESS and could also result in large $\hat{R}$ if the chains are moving very slowly through the parameter space—by chance, they may not be well mixed unless we sample for a very long time.

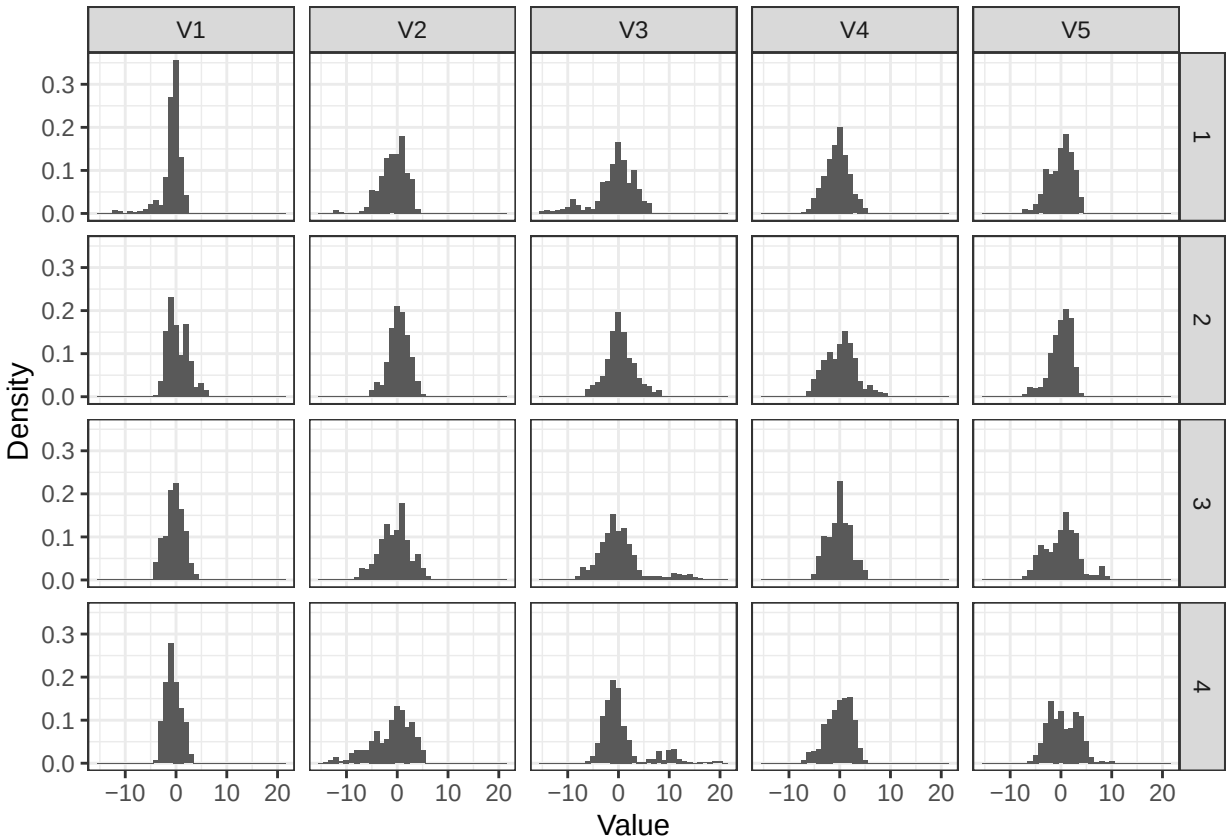
Indeed, if we increase the number of iterations to 10,000, $\hat{R}$ looks considerably better, although still not perfect:

Dimension	Rhat	ESS
V1	1.027	140.486
V2	1.031	107.725
V3	1.024	107.227
V4	1.107	31.512
V5	1.047	102.044

Looking at the trace plots for 1000 iterations confirms the hypothesis that the chains are badly autocorrelated:



We can also look at density plots, separated by chain, to see if the chains converge on roughly similar estimates. Below, each columns shows a dimension (V1-V5) and each row shows an MCMC chain (1-4).

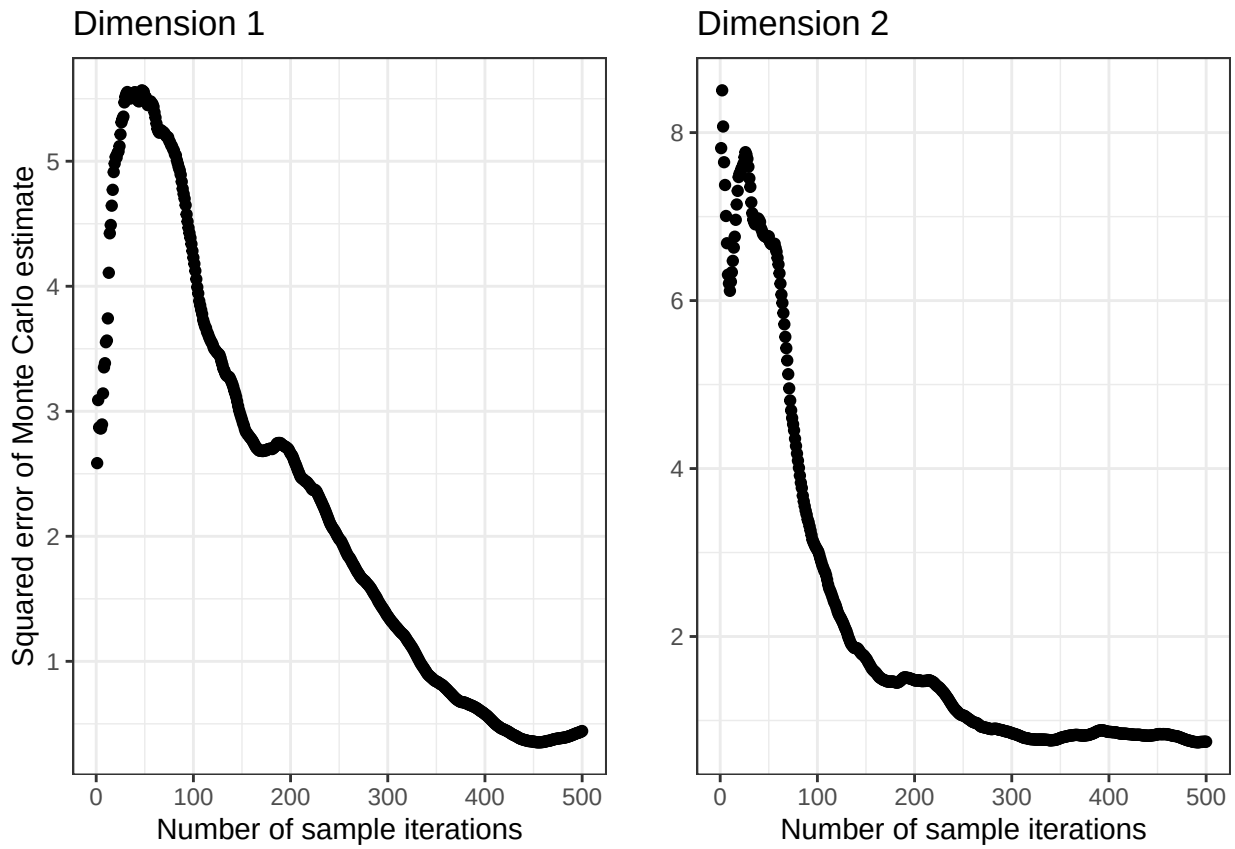


The chains seem to have similar central tendency, but the shape of their sampled distributions are not the same. In many cases they don't look particularly Gaussian, which is not surprising given our very small ESS.

d) Squared error of Monte Carlo mean estimates

$MSE_i = \frac{1}{C-1}\sum_{c=1}^{C}[(\frac{1}{N}\sum_{n=1}^{N} x_{i,n,c}) - \mu_i]^2$

where C denotes the number of chains and N denotes the sampling (not warmup!) iterations per chain, i denotes the dimension of the target distribution, and $x_{i,n,c}$ denotes the value of the n_{th} sample from the c_{th} chain in the i_{th} dimension.



This value appears to generally decrease, but with oscillatory behaviour. This is likely because the (highly autocorrelated) chains are wandering slowly along their own trajectories, and we have not let them run for long enough to properly mix. When chains move on diverging trajectories the MSE increases, and when they start to move towards eachother again, the MSE decreases.

e) Optimizing (static) L

If we maintain a stepsize of $\epsilon = 0.05$, $L^* \approx 149$ for this target distribution ($max(\Sigma_{ii}) = 8.85$) When running my sampler with $L = L^*$, I received a warning from the function `ess_bulk` in the package `posterior`: `Warning: The ESS has been capped to avoid unstable estimates.` I checked the Stan forums and found that this can be a sign of "antithetic sampling," which I interpret as "the samples are very negatively autocorrelated." While this can improve the central qualities of the estimator, I also read that it can reduce the tail effective sample size. I have therefore reported both bulk and tail effective sample sizes below.

Dimension	L	Rhat	ESS_Bulk	ESS_Bulk_Per_Grad	ESS_Tail
V1	10	1.3612	9.5088	0.0002	27.4560
V2	10	1.4172	8.8215	0.0002	39.7756
V3	10	1.2555	11.7763	0.0003	25.0246
V4	10	1.1976	16.1176	0.0004	41.4980
V5	10	1.3248	9.9019	0.0002	33.3271
V1	80	1.0002	1404.7603	0.0043	1775.0210
V2	80	1.0025	892.7963	0.0028	1361.1814
V3	80	1.0086	753.1625	0.0023	1438.3982
V4	80	1.0019	992.4819	0.0031	1352.3086
V5	80	1.0020	839.5510	0.0026	1600.2366
V1	149	1.0839	6602.0600	0.0110	207.6902

Dimension	L	Rhat	ESS_Bulk	ESS_Bulk_Per_Grad	ESS_Tail
V2	149	1.0405	6602.0600	0.0110	259.9528
V3	149	1.0023	6602.0600	0.0110	1934.1218
V4	149	1.0012	6602.0600	0.0110	1792.0942
V5	149	1.0047	6587.9326	0.0110	1772.9138

Indeed we can see that for two of the dimensions (V1 and V2), the "optimized" L^* results in lower values of `ESS_Tail` and $\hat{R} \geq 1.01$ (note that V1 is always bad, V2 is only bad for some runs–so when I knit this markdown document you may see fine diagnostics for V2).

The Stan forum post suggested either running for more iterations (antitheticity tends to occur more frequently on short chains?) or capping the `max_treedepth` (not applicable for this sampler).

If I double the number of iterations (N = 2000), the same warning arises and $\hat{R}$ is still borderline:

Dimension	L	Rhat	ESS_Bulk	ESS_Bulk_Per_Grad	ESS_Tail
V1	149	1.0903	14408.24	0.0120	66.2358
V2	149	1.0356	14408.24	0.0120	295.7658
V3	149	1.0109	12862.72	0.0107	2684.1460
V4	149	0.9995	14408.24	0.0120	3324.5109
V5	149	1.0056	13268.60	0.0111	3667.7240

While we can't adapt `max_treedepth` here, that is effectively the NUTS workaround for controlling/capping `L`, which suggests to me that this `L` value may also not be optimal (or my sampler is totally dysfunctional, since this is supposed to be the optimal value…yikes).

f) A new HMC function for jittering L per iteration

```
HMC_jitter = function(log_p,
               grad_log_p,
               theta0,
               n_iter,
               eps,
               Lmax){
  q = theta0
  draws = matrix(NA, nrow = n_iter, ncol = length(theta0))
  numgrad = numeric(n_iter)

  for (n in 1:n_iter){
  L = sample(1:Lmax, 1)
  numgrad[n] = 0
  start_q = q
  p = rnorm(length(q),0,1)
  start_p = p

  ################ Leapfrog ##############################
  # half step for momentum at the beginning
  p = p + eps*grad_log_p(q) / 2
  numgrad[n] = numgrad[n] + 1

  for (i in 1:L){
```

```r
  # full step for position
    q = q + eps * p
  # full step for momentum, except at end of trajectory
    if (i!=L) {
      p = p + eps*grad_log_p(q)
      numgrad[n] = numgrad[n] + 1
      }
  }

  # half step for momentum at the end.
  p = p + eps*grad_log_p(q) / 2
  numgrad[n] = numgrad[n] + 1
  p = -p # for detail balance, not strictly necessary
  ##########################################################

  ################## Metropolis Acceptance ################
  # evaluate potential and kinetic energies
  start_U = -log_p(start_q)
  start_K = sum(start_p^2) / 2
  proposed_U = -log_p(q)
  proposed_K = sum(p^2) / 2

  # accept or reject the state at end of trajectory
  if (runif(1) < exp(start_U-proposed_U+start_K-proposed_K)){
    draws[n,1:length(q)] = q
    } else{
      q = start_q
      draws[n,1:length(q)] = q
    }
  ##########################################################
  }
  return(list(draws, numgrad))
}
```

Using the updated function and $L_{max} = L^*$ I return the following summary statistics:

Dimension	Rhat	ESS_Per_Iter
V1	1.004	0.004
V2	1.003	0.004
V3	1.004	0.003
V4	1.004	0.004
V5	1.001	0.003

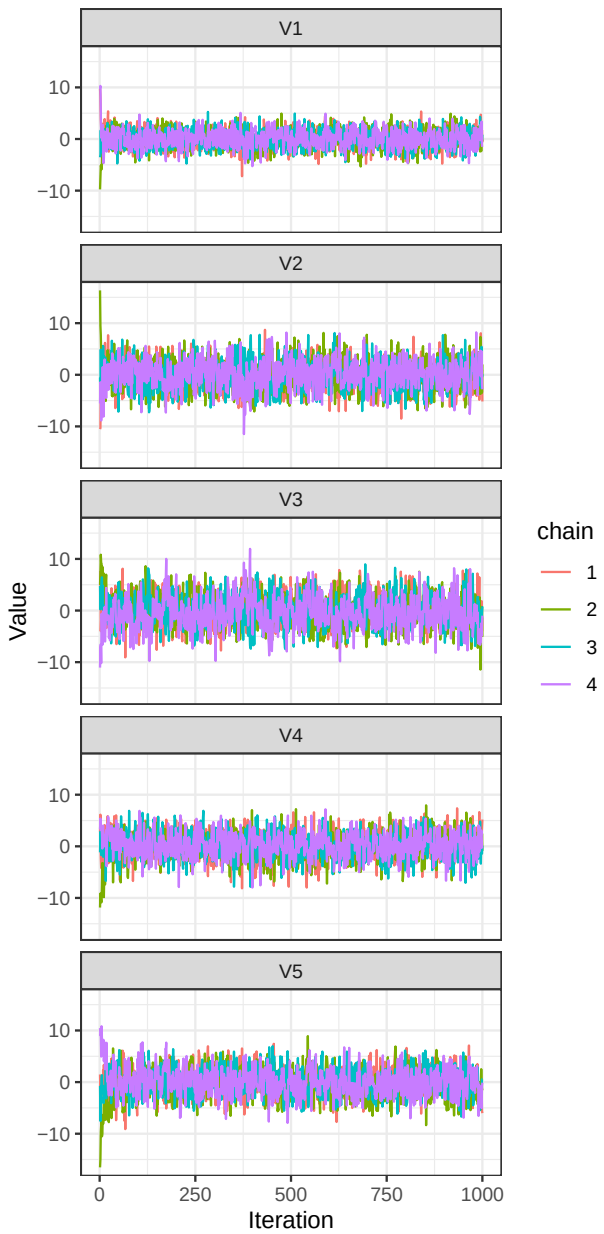
Here the issues with antitheticity are resolved, supporting the use of a jittering the value of `L`. We can see however, that the bulk ESS per gradient evaluation is lower than for $L = L^*$. Perhaps a nice compromise would be to jitter L for values $\{\frac{L^*}{2}...L^*\}$ to avoid very small values of L.

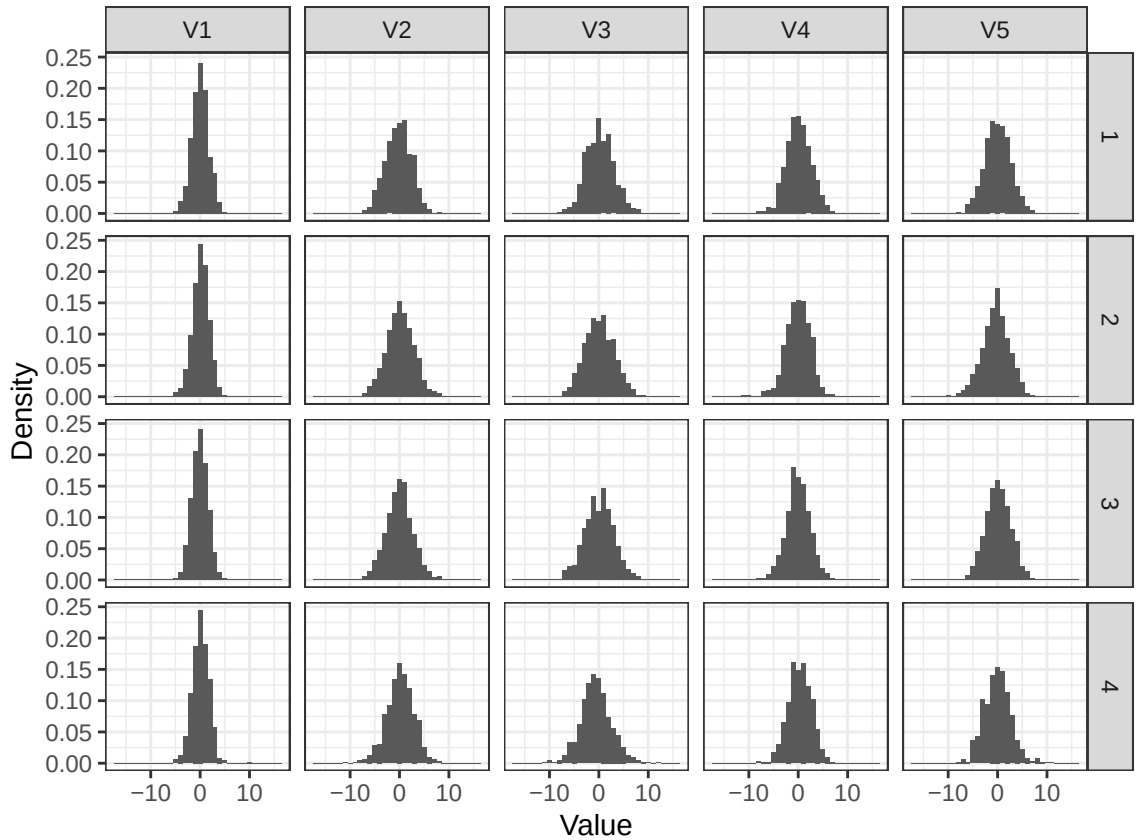
Full diagnostics for L = 80, eps = 0.05, iterations = 1000 (without jittering)

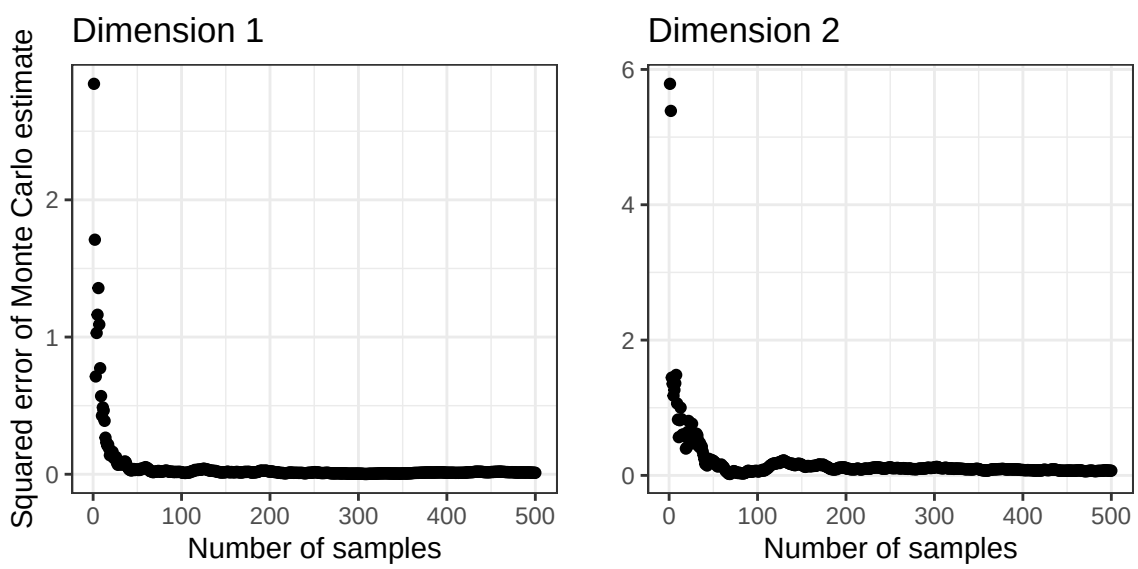
variable	mean	median	sd	q5	q95	mcse_mean
X1	-0.001	0.006	1.659	-2.660	2.741	0.072

variable	mean	median	sd	q5	q95	mcse_mean
X2	-0.025	-0.111	2.667	-4.304	4.312	0.133
X3	-0.128	-0.128	2.949	-4.798	4.634	0.146
X4	0.135	0.232	2.407	-4.102	3.743	0.112
X5	0.026	-0.027	2.629	-4.186	4.434	0.152

Dimension	Rhat	ESS
V1	1.003	952.507
V2	1.006	974.468
V3	1.011	729.296
V4	1.005	792.180
V5	1.007	641.636







Exercise 3

a) Show that $KL(q||p) \geq 0$ and $KL(q||p) = 0$ if and only if $p = q$

Let $KL(q||p) = \int (log(q(z)) - log(p(z)))q(z)dz$

Jensen's inequality for concave functions states that $f(\mathbb{E}(x)) \geq \mathbb{E}(f(x))$. We can therefore see that

$\int q(z)log(\frac{p(z)}{q(z)})dz \leq log(\int q(z)\frac{p(z)}{q(z)}dz) = log(1) = 0$

Flipping the terms in the logarithm of the first integral gives

$-\int q(z)log(\frac{q(z)}{p(z)})dz \leq 0$

which is equivalent to

$KL(q||p) = \int q(z)log(\frac{q(z)}{p(z)})dz \geq 0$

For Jensen's inequality, equality is only achieved when x (from the term $f(\mathbb{E}(x)) \geq \mathbb{E}(f(x))$) is constant "almost everywhere." I believe this is because in the continuous case, we can change a single point in the probability density function but it will contribute "measure 0" to the probability—you mentioned this in lecture, I think. Having not taken coursework in measure theory this seems like a fuzzy rule to me…if we have an infinite number of points on the line, couldn't we then change the pdf at a smaller infinity of those points and still arrive at the same conclusion, since each point contributes "measure 0"? But I digress. The point is that we want $x = c$, where c is some constant. In this case, that means $\frac{p(z)}{q(z)} = c$, from which we can say

$\int p(z)dz = \int cq(z)dz$

Since both $p(z)$ and $q(z)$ are probability density functions, they integrate to 1, yielding $c = 1$. So we achieve equality ($KL(q||p) = 0$) if and only if $p(z) = q(z)$.

b) KL for multivariate Gaussians

First, we can rewrite $KL(q||p)$ as $KL(q||p) = \mathbb{E}_q(log(q(z)) - log(p(z))$. For p, a d-dimensional Gaussian with mean μ and non-diagonal covariance matrix Σ, and q, a d-dimensional Gaussian with mean ν and diagonal covariance matrix ψ, we can substitute the logarithm of the probability density function for a multivariate Gaussian and arrive at:

$KL(q||p) = \mathbb{E}_q(-\frac{1}{2}log(|\psi|) + \frac{1}{2}log(|\Sigma|) - \frac{1}{2}(x-\nu)^T\psi^{-1}(x-\nu) + \frac{1}{2}(x-\mu)^T\Sigma^{-1}(x-\mu))$

A convenient trick (thanks, Google!) is that $\mathbb{E}_q((x-a)^TA(x-a)) = tr(\psi A) + (\nu - a)^TA(\nu - a)$.

For the term $(\frac{1}{2}(x-\nu)^T\psi^{-1}(x-\nu))$ this simplifies to $tr(\psi\psi^{-1}) + (\nu-\nu)^T\psi^{-1}(\nu-\nu) = tr(\mathbf{I}) = d$ (for a d-dimensional identity matrix).

For the term $\frac{1}{2}(x-\mu)^T\Sigma^{-1}(x-\mu)$, this becomes $tr(\psi\Sigma^{-1}) + (\nu-\mu)^T\Sigma^{-1}(\nu-\mu)$, where $tr(A) = \sum_{i=1}^d A_{ii}$.

Together this yields

$KL(q||p) = \frac{1}{2}log(|\Sigma|) + d - \frac{1}{2}log(|\psi|) + (\nu-\mu)^T\Sigma^{-1}(\nu-\mu) + \sum_{i=1}^d(\Sigma^{-1})_{ii}\psi_{ii}$

The first two terms can be combined into a constant, K, which does not depend on ν or ψ. The final form is:

$KL(q||p) = K - \frac{1}{2}log(|\psi|) + (\nu-\mu)^T\Sigma^{-1}(\nu-\mu) + \sum_{i=1}^d(\Sigma^{-1})_{ii}\psi_{ii}$

(As discussed previously, the sign on $\frac{1}{2}log(|\psi|)$ should be negative, and I believe you may have also dropped the inverse on Σ in the middle term.)

c) Show that the variational parameters which minimize $KL(q||p)$ are $\nu = \mu$ and $\psi_{ii} = \frac{1}{(\Sigma^{-1})_{ii}}$

To show this we can take the gradient of $KL(q||p)$ with respect to ν and ψ, set these equal to 0, and solve for ν or ψ. We can then show that the second order gradient is positive to verify that the critical points are minima.

For the gradient with respect to ν, the term we care about is $(\nu-\mu)^T\Sigma^{-1}(\nu-\mu)$. We can take the gradient using the product rule. In full disclosure I hadn't the faintest idea how to do this, but Stack Exchange came to the rescue. I'll write out the product rule bit below, but the punchline is that for $f = x^TAx$, $\nabla f = (A^T + A)x$.

To arrive at this rule, let $g(x) = x^T$, $h(x) = Ax$, and $f(x) = g(x)h(x)$. Then by product rule,

$df(x) = dg(x)h(x) + g(x)dh(x)$

where $dg(x) = d(x^T) = (dx)^T$ and $dh(x) = Adx$. Substituting, we get

$df(x) = (dx)^T Ax + x^T Adx$

Evidently, $(dx)^T Ax = x^T A^T dx$ (I lack intuition for this but can convince myself that it's true for a 2x2 matrix), so:

$df = x^T A^T dx + x^T Adx = x^T (A^T + A)dx$

By definition, $df = (\nabla f)^T$ so $\nabla f = (A^T + A)x$

With this trick in hand, we can go on to take the gradient wrt ν. If

$f(\nu) = (\nu - \mu)^T \Sigma^{-1} (\nu - \mu)$

then

$\nabla f_v = \frac{1}{2}((\Sigma^{-1})^T + \Sigma^{-1})(\nu - \mu)$

which simplifies to $(\Sigma^{-1})(\nu - \mu)$ because Σ^{-1} is symmetrical.

Because Σ^{-1} is positive-definite, the gradient will only be equal to 0 when $\nu = \mu$. We can check that this is a minima by computing the second order gradient of the expression ($\nabla^2 f_v = \Sigma^{-1}$). According to Google this is also called the "Hessian" matrix (another word which I could perhaps have defined in 2017, but deleted from my brain to make space for bumblebee facts…). Because it is positive definite, the function has positive curvature everywhere. This means $\nu = \mu$ is a minimum.

Now, we can turn our attention to ψ, which is (much to my relief) a bit simpler to work with. The terms we care about are:

$-\frac{1}{2} log(|\psi|) + \sum_{i=1}^{d} (\Sigma^{-1})_{ii} \psi_{ii}$

which are equivalent to:

$-\frac{1}{2} log(\Pi_{i=1}^{d} \psi_{ii}) + \sum_{i=1}^{d} (\Sigma^{-1})_{ii} \psi_{ii}$

$= -\frac{1}{2} \sum_{i=1}^{d} log(\psi_{ii}) + \sum_{i=1}^{d} (\Sigma^{-1})_{ii} \psi_{ii}$

For each ψ_{ii},

$\frac{\delta f}{\delta \psi_{ii}} = -\frac{1}{2}(\frac{1}{\psi_{ii}}) + \frac{1}{2}(\Sigma^{-1})_{ii} = \frac{1}{2}((\Sigma^{-1})_{ii} - \frac{1}{\psi_{ii}})$

which is equal to 0 when $\psi_{ii} = \frac{1}{(\Sigma^{-1})_{ii}}$.

The second derivative $\frac{\delta^2 f}{\delta \psi_{ii}} = \frac{1}{\psi_{ii}}$, which is always positive, indicating that $\psi_{ii} = \frac{1}{(\Sigma^{-1})_{ii}}$ is also a minimum.

d) VI underestimates the variance of p

To show that $Var(z_i|z_{-i}) = \frac{1}{(\Sigma^{-1})_{ii}}$ we will start from:

$log(p(z)) = K - \frac{1}{2}(z - \mu)^T \Sigma^{-1} (z - \mu)$

We know also that

$log(p(z_i|z_{-i})) = K - \frac{1}{2} \frac{(z_i - f(z_{-i}))^2}{Var(z_i|z_{-i})} = K - \frac{1}{2} \frac{z_i^2}{Var(z_i|z_{-i})} + \frac{f^2(z_{-i})}{Var(z_i|z_{-i})} - \frac{2z_i f(z_{-i})}{Var(z_i|z_{-i})}$

Therefore, if we can expand $log(p(z_i|z_{-i}))$ and isolate the *only* term containing z_i^2, we will know immediately that the coefficient on this term is equal to $-\frac{1}{2Var(z_i|z_{-i})}$

To expand $log(p(z_i|z_{-i}))$, we recall that $p(z) = p(z_i|z_{-i})p(z_{-i})$, or equivalently, $log(p(z)) = log(p(z_i|z_{-1})) + log(p(z_{-i})))$.

Returning to

$log(p(z)) = K - \frac{1}{2}(z-\mu)^T\Sigma^{-1}(z-\mu)$

For now, let's set aside the $-\frac{1}{2}$. We can expand the expression by considering that $(z-\mu)^T$ is a 1xn vector (call it u^T) and $\Sigma^{-1}(z-\mu)$ is an nx1 vector (call it ν, with indices k).

$\nu_k = \sum_j (\Sigma)^{-1}_{kj}(z_j - \mu_j)$

Then:

$u^T\nu = \sum_k u_k\nu_k = \sum_k (z_k - \mu_k)\sum_j (\Sigma)^{-1}_{kj}(z_j - \mu_j)$

The is evident that the term we are looking for (containing z_i^2) will only be achieved when $k = i$ and $j = i$. Setting these values gives us:

$(z_i - \mu_i)^2(\Sigma)^{-1}_{ii})$

such that the coefficient on the z_i^2 term will be $(\Sigma)^{-1}_{ii}$. Recall that we set aside a $-\frac{1}{2}$ earlier. Reintroducing it, we achieve:

$-\frac{1}{2Var(z_i|z_{-i})} = -\frac{1}{2}(\Sigma)^{-1}_{ii}$

or more simply, $Var(z_i|z_{-i}) = \frac{1}{(\Sigma)^{-1}_{ii}}$

Next, we want to show that $\psi_{ii} \leq \Sigma_{ii}$. From part (c), we know that the VI estimate for ψ_{ii} which minimizes the KL divergence is $\psi_{ii} = \frac{1}{(\Sigma^{-1})_{ii}}$.

The law of total variance tells us that:

$Var(z_i) = \mathbb{E}[Var(z_i|z_{-i})] + Var(\mathbb{E}[z_i|z_{-i}])$

We have just computed the first term, namely:

$Var(z_i) = \frac{1}{(\Sigma)^{-1}_{ii}} + Var(\mathbb{E}[z_i|z_{-i}])$

The second term will always be ≥ 0, as it is a variance.

But of course, $\frac{1}{(\Sigma)^{-1}_{ii}} = \psi_{ii}$ and $Var(z_i) = \Sigma_{ii}$, so:

$\Sigma_{ii} = \psi_{ii} + Var(\mathbb{E}[z_i|z_{-i}])$

This means that $\Sigma_{ii} > \psi_{ii}$ *except* in the case where Σ is a diagonal matrix, in which case $\mathbb{E}[z_i|z_{-i}]$ is a constant (e.g., z_{-i} gives us no information about z_i) and the full term $Var(\mathbb{E}[z_i|z_{-i}])$ goes to 0 (leaving $\Sigma_{ii} = \psi_{ii}$).

When Σ is not a diagonal matrix, then at least one value Σ_{ij} must be nonzero. In this case, $Var(\mathbb{E}[z_i|z_j]) > 0$, and likewise $Var(\mathbb{E}[z_j|z_i]) > 0$, meaning that both $\Sigma_{ii} > \psi_{ii}$ and $\Sigma_{jj} > \psi_{jj}$.

GenAI Use Statement: I did not use AI to generate the code or writing in this assignment. However, I did use it to walk me through several computations/derivations, particularly for Exercise 3. At this point I could explain those derivations (possibly without looking at my notes…?) but admittedly this method did not grant me a full understanding of all of the theory. Most likely, nothing short of an entire undergraduate degree's worth of mathematics courses would grant me that understanding, and I simply could not achieve that in two weeks (sorry!).